

Reflection Log

Noorinder

Credit 3

Assignment Name: Vehicle, Car, Truck, Minivan, VehicleTest

How has your program changed from planning to coding to now?

At first, the program was planned as a simple demonstration of inheritance and polymorphism using vehicles as a theme. The base class `Vehicle` was designed to store common attributes like make, model, and year, while subclasses `Car`, `Truck`, and `Minivan` added specific details such as the number of doors, payload capacity, and seating capacity. The `VehicleTest` class was responsible for displaying vehicle details interactively.

1. Missing Override Annotation

- **Problem:** Initially, the `getDetails()` method in the subclasses didn't have the `@Override` annotation. Although the program worked, this omission reduced clarity about the method's relationship to the abstract method in `Vehicle`.
- **Fix:** I added the `@Override` annotation to the `getDetails()` method in all subclasses.

Reflection: This change improved code readability and made it clear that the subclasses were fulfilling the requirements of the abstract class.

Code Example:

```
@Override
public String getDetails() {
    return String.format("Car: %s, Doors: %d", super.toString(), numDoors);
}
```

2. Handling Invalid Vehicle Type

- **Problem:** Entering an invalid vehicle type number caused the program to display incorrect behavior or enter an infinite loop. This was due to inadequate handling of invalid inputs in the `switch` statement.
- **Fix:** I added a `default` case to handle invalid inputs and prompt the user again.

Reflection: Adding input validation made the program more user-friendly and resilient against unexpected inputs.

Code Example:

```
switch (vehicleType) {
    case 1:
        vehicle = car;
        break;
    case 2:
        vehicle = truck;
        break;
    case 3:
        vehicle = minivan;
        break;
    default:
        System.out.println("Invalid Vehicle type! Please try again.");
        continue; // Re-prompts the user
}
```

3. Lack of Validation for Vehicle Year

- **Problem:** The program allowed invalid year values (e.g., negative or future years) when creating vehicle instances.
- **Fix:** I added validation in the `Vehicle` constructor to ensure the year is within a realistic range (e.g., greater than 1885 and not beyond the current year).

Reflection: This adjustment ensured data integrity and prevented unrealistic entries.

Code Example:

```
public Vehicle(String make, String model, int year) {
    if (year < 1886 || year > 2024) { // Validate year
        throw new IllegalArgumentException("Invalid year for a vehicle.");
    }
    this.make = make;
    this.model = model;
    this.year = year;
}
```

4. Incomplete Vehicle Details

- **Problem:** The `getDetails()` method in `Minivan` didn't include the seating capacity when displaying vehicle information. This caused the output to be incomplete for minivans.
- **Fix:** I updated the `getDetails()` method in `Minivan` to include seating capacity in the returned string.

Reflection: This fix improved the output consistency across all vehicle types.

Code Example:

```
@Override
public String getDetails() {
    return String.format("Minivan: %s, Seating Capacity: %d", super.toString(), seatingCapacity);
}
```

5. No Quit Confirmation

- **Problem:** The program exited immediately when the user selected the quit option without any confirmation. This could lead to accidental exits.
- **Fix:** I added a confirmation message asking the user to confirm before exiting.

Reflection: This change made the program more interactive and reduced the chances of accidental termination.

Code Example:

```
if (action.equalsIgnoreCase("Q")) {
    System.out.print("Are you sure you want to quit? (yes/no): ");
    String confirmation = input.next();
    if (confirmation.equalsIgnoreCase("yes")) {
        break; // Exit the loop
    }
}
```